UNIT-I — INTRODUCTION TO DISTRIBUTED SYSTEMS (Complete Notes)

Labels: 📘 official syllabus · 📝 assignment-emphasized · ➕ additional/exam-oriented

---

1. WHAT IS A DISTRIBUTED SYSTEM? 📘📝

Definition

A **distributed system** is a system in which **hardware or software components located at networked computers communicate and coordinate their actions only by passing messages** (Coulouris et al.). Another common form: *"A collection of independent computers that appears to its users as a single coherent system"* (Tanenbaum).

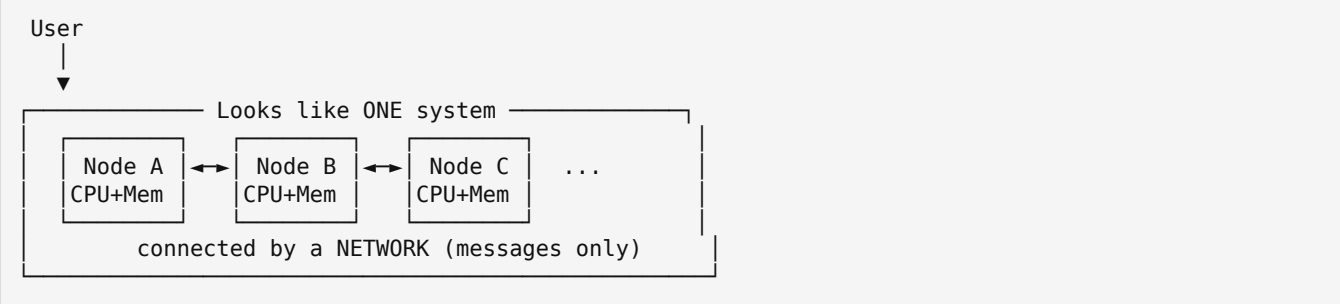
Simple explanation (🗣️ Hinglish)

Bahut saare alag-alag computers network se jude hain aur milkar ek kaam karte hain, lekin user ko lagta hai ki ek hi system hai. Jaise **Google Search** – aapko ek search box dikhta hai, peeche hazaaron computers kaam kar rahe hote hain.

Detailed explanation

- Each computer (node) has its **own CPU, memory, OS clock**. There is **no shared memory** and **no shared clock**.
- Nodes cooperate **only via messages** over a network (LAN/WAN/Internet).
- The system hides the complexity → the user sees "one computer" (this is called **transparency**).
- Examples: the Web, ATM/banking network, email, online multiplayer games, Google Search, Netflix, UPI payments, cloud platforms.

Diagram



Advantages / Disadvantages (short)

Advantages	Disadvantages
Resource sharing, scalability, reliability	Complex design, hard debugging

Advantages	Disadvantages
Better performance through parallelism	Network delays and failures
Geographic distribution	Security risks; no global clock

Exam points

- Key phrases: *autonomous computers, network, message passing, coordination, single coherent system*.
- Motivation: **resource sharing**.

🧠 Remember This

> DS = independent computers + network + only message passing + user sees one system.

---

2. CHARACTERISTICS OF DISTRIBUTED SYSTEMS 📘📝 (A1-1a)

Coulouris lists **three core characteristics**; textbooks add others (➕ marked).

#	Characteristic	Meaning	Example
1	**Concurrency**	Many components execute at the same time; they share resources (files, DB)	Many users booking tickets at IRCTC together
2	**No global clock**	Each computer has its own clock; clocks drift; only message passing can coordinate time	Two servers show different times → ordering problem (→ Unit-II)
3	**Independent failures**	Any node or network link can fail without others failing; failure of one may not be known to others	A server crashes but other servers still run
4 ➕	**Resource sharing**	Hardware/software/data shared across network	Shared printers, cloud storage
5 ➕	**Heterogeneity**	Different networks, hardware, OS, languages, implementations; hidden by *middleware*	Android phone talking to Linux server
6 ➕	**Openness**	System can be extended/re-implemented using published interfaces	Web standards (HTTP, REST)
7 ➕	**Scalability**	Works well when users/resources increase	YouTube
8 ➕	**Fault tolerance / Failure handling**	Detect, mask, tolerate failure, recover, use redundancy	RAID, replicated DB
9 ➕	**Transparency**	Hide distribution from user (see below)	Google Drive file looks local

#	Characteristic	Meaning	Example
10 ➕	**Security**	Confidentiality, integrity, availability	HTTPS, authentication
11 ➕	**Quality of Service (QoS)**	Reliability, performance, security, adaptability	Video streaming without buffering

Types of Transparency ➕ (ISO/RM-ODP; Coulouris)

Type	Hides
Access	Differences in data representation & how resource is accessed
Location	Where the resource physically is (URL doesn't reveal server location)
Concurrency	That many users share a resource simultaneously
Replication	That multiple copies exist
Failure	Faults and recovery
Mobility	Movement of resources/clients
Performance	Reconfiguration to improve performance
Scaling	System growth without changing structure

Challenges (➕ Coulouris): heterogeneity, openness, security, scalability, failure handling, concurrency, transparency, quality of service.

Exam points

- If asked "Define and discuss characteristics with examples" (2–3 marks): definition + **the 3 core ones** + 2–3 more + one example each.
- Examiner favourite: **"no global clock"** and **"independent failure"** (they cause Unit-II problems).

🧠 Remember This

> Concurrency · No global clock · Independent failures (core 3) + heterogeneity, openness, scalability, transparency, fault tolerance, security.

---

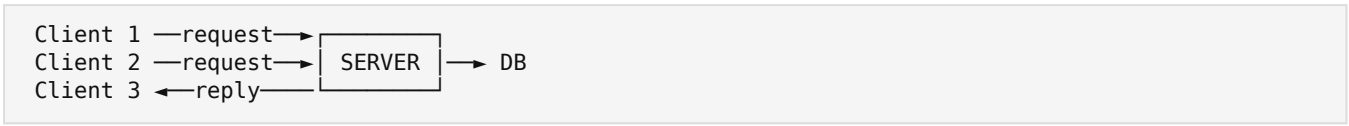
3. EXAMPLES OF DISTRIBUTED SYSTEMS 📘📝 (A1-1b)

3.1 Client–Server

Definition: An architecture where **client** processes send requests to **server** processes, which manage resources and reply.

Simple (🗣️): Aap browser (client) se website maangte ho, web server data bhejta hai. Server hamesha "on" rehta hai, client jab chahe request karta hai.

Working (steps) 1. Client sends request (e.g., HTTP GET). 2. Server receives, processes (maybe queries DB). 3. Server sends response. 4. Client displays result.



Examples: Web browser–web server, email (SMTP/IMAP), ATM–bank server, file server (NFS), database server. **Advantages:** central control, easy security & backup, easy to manage, clear roles. **Disadvantages:** server = **single point of failure**, bottleneck under heavy load, costlier to scale, network dependent. **Variants:** 2-tier, 3-tier (presentation–application–data), n-tier.

3.2 Peer-to-Peer (P2P)

Definition: Every node (**peer**) acts as **both client and server**; peers share resources directly without a central server.

Simple (🗣️): Sab barabar hain. Torrent me aap file download bhi karte ho aur dusron ko upload bhi.



Types: *Centralized-index* (Napster), *pure/unstructured* (Gnutella), *structured with DHT* (Chord, Kademlia) ➕. **Examples:** BitTorrent, Blockchain/Bitcoin nodes, Skype (early), IPFS. **Advantages:** no single point of failure, scales as more peers join, cost-effective, resource utilisation. **Disadvantages:** hard to secure/control, hard searching/management, uneven peer availability, copyright/legal issues.

3.3 Grid Computing

Definition: A system that **coordinates loosely-coupled, heterogeneous, geographically dispersed computers** (belonging to different organisations) to work as a **virtual supercomputer** for large tasks.

Simple (🗣️): Alag-alag jagah ke computers ka idle power jod kar ek bahut bada kaam (jaise weather simulation) karna. Jaise bijli ka *power grid* – socket lagao, power lo.



Examples: SETI@home, CERN LHC Computing Grid, Folding@home, weather/genome projects. **Features:** heterogeneous resources, multiple administrative domains, batch-oriented,

resource discovery & scheduling, uses **Globus toolkit** ➕. **Advantages:** huge computing power at low cost, uses idle resources, scalable, reliability via redundancy. **Disadvantages:** complex middleware, security & trust across organisations, resource scheduling difficulty, slow WAN links.

3.4 Cloud Computing (only as an *example* of a DS here)

Definition: Delivery of computing services (servers, storage, DB, software) **over the Internet on a pay-per-use, on-demand basis** with rapid elasticity (NIST-type definition ➕).

Simple (🗣️): Apna server rakhne ke bajaye AWS/Azure/Google se kiraye pe computer lena – jitna use, utna paisa.

```
Users ──Internet──► [ Cloud Provider Data Centres ]
                     IaaS | PaaS | SaaS (service models)
                     virtualised, shared pool of resources
```

Key features: on-demand self-service, broad network access, resource pooling (virtualisation), rapid elasticity, measured service. **Examples:** AWS, Microsoft Azure, Google Cloud, Gmail/Google Docs (SaaS). **Advantages:** no upfront cost, elastic scaling, high availability, easy access anywhere. **Disadvantages:** internet dependence, data privacy/vendor lock-in, recurring cost, less control.

> ⚠️ **Outside Mid-Sem Syllabus:** service models (IaaS/PaaS/SaaS), deployment types, migration – Unit-IV. Just mention them in one line if asked in Unit-I.

3.5 COMPARISON: Client–Server vs P2P vs Grid vs Cloud 📝

Feature	Client-Server	Peer-to-Peer	Grid Computing	Cloud Computing
Basic idea	Clients request, servers serve	All nodes equal (client+server)	Pool many organisations' resources as virtual supercomputer	On-demand services over Internet
Control	Centralised (server)	Decentralised	Decentralised/virtual organisation, scheduler	Centralised by provider (data centres)
Roles	Fixed	Dynamic, symmetric	Resource provider / consumer	Provider / customer
Main purpose	Service access, resource sharing	File/resource sharing	Solve huge computational problems	Rent IT services (compute, storage, apps)
Scalability	Limited by server	High	Very high	Very high (elastic)
Failure	Server = SPOF	No SPOF	Job rescheduling on failure	Redundancy across data centres
Ownership	One organisation	Individual users	Multiple organisations	Cloud provider
Resource type	Homogeneous mostly	Heterogeneous	Heterogeneous, distributed	Virtualised, homogeneous pool

Feature	Client-Server	Peer-to-Peer	Grid Computing	Cloud Computing
Billing	Usually none/fixed	Free	Usually project/sharing based	**Pay-per-use**
Typical use	Web, e-mail, DB	BitTorrent, blockchain	SETI@home, CERN	AWS, Azure, Gmail
Security	Easier to control	Hard	Hard (trust across domains)	Provider-managed, shared responsibility

Exam point: A 2-mark answer = one-line each + 1 application each; a 5-mark answer = table + diagrams.

🧠 Remember This

> CS = central server · P2P = everyone equal · Grid = many orgs share compute for big jobs · Cloud = rent services pay-per-use.

---

4. ADVANTAGES OF DISTRIBUTED SYSTEMS 📘

Advantage	Explanation
Resource sharing	Printers, files, data, CPUs shared over network
Scalability	Add more nodes to handle more load (horizontal scaling)
Reliability/Availability	One node failure doesn't stop the whole system (replication)
Performance/Speed	Parallel execution of tasks, load distribution
Economics	Many cheap PCs give better price/performance than one mainframe
Geographical distribution	Users/data near each other → low latency (CDNs)
Incremental growth	Add capacity gradually
Openness/Flexibility	Mix hardware/software from different vendors
Fault tolerance	Redundancy hides failures

Disadvantages (limitations): complexity, network dependence, security threats, no global clock/state, difficult debugging, partial failure, higher software cost, coordination overhead.

🧠 Remember This

> "Share, Scale, Survive (reliability), Speed, Save (cost)".

---

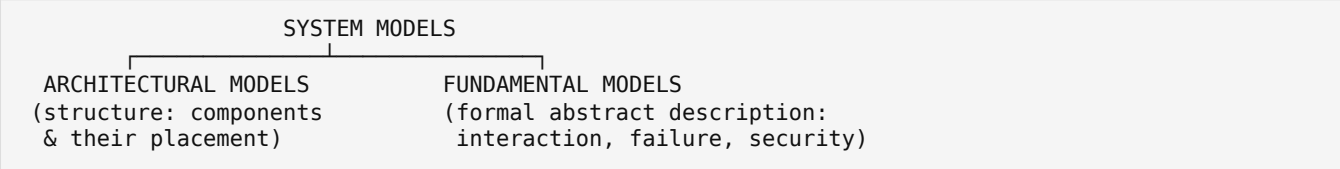
5. SYSTEM MODELS 📘📝 (A1-2a)

5.1 Introduction – need for system models

Definition: A **system model** is an **abstract, simplified description** of the relevant aspects of a distributed system (its components, their interactions, and the properties/assumptions about failures and security).

Why we need them (🗣️: "naksha banake samajhna"): 1. To show the **common properties & design issues** of DS. 2. To make **explicit assumptions** (e.g., messages may be lost, clocks drift). 3. To reason about **what is possible / impossible** and **prove** algorithm correctness. 4. To generalise – an algorithm proved under a model works in all systems satisfying that model. 5. Helps compare systems and decide design.

Classification:



5.2 ARCHITECTURAL MODELS

Definition: Describe the **structure of the system in terms of components, their placement on network, and their relationships/roles**. Ideas: *placement of components*, *relationship (client-server, peer)*.

(a) Software layers & Middleware



- **Platform:** lowest layers (hardware + OS), e.g., Intel x86 + Windows/Linux.
- **Middleware:** software layer that **masks heterogeneity** and gives a uniform programming model (RMI, RPC, CORBA). 🗣️ "Beech ka pul".

(b) System architectures (main styles)

1. **Client–server model** – see §3.1. (Diagram above)
2. **Multiple servers** – a service is provided by **several server processes** on different hosts through **partitioning** (each holds part of data, e.g., web servers) or **replication** (copies for fault tolerance, e.g., DNS). `Client → Server 1 ┐ Client → Server 2 ┼ cooperate to provide one service Client → Server 3 ┘`
3. **Proxy servers and caches** – **cache** stores recently used objects near clients; **proxy** shares cache for many clients. Reduces latency & network load. (browser cache, web proxy, CDN) `Client → Proxy/Cache → Web server`
4. **Peer processes (P2P)** – all processes play similar roles (§3.2).

5. **Mobile code** – code is downloaded to client and runs locally (Java applets, JavaScript). Pros: interactive/faster; cons: security risk.
6. **Mobile agents** – running program (code+data) that **travels** between computers doing work on behalf of user. Pros: reduces communication; cons: security/tracking.
7. **Thin clients / network computers** – minimal local processing; heavy work on server (cloud desktops, Chromebook).
8. **Mobile devices & spontaneous networking** ➕ – phones, IoT discover each other (Bluetooth).

(c) Layered / tier architecture (➕ common exam answer)

Tier	Role
Presentation	UI
Application (logic)	Business rules
Data	Database
- *Layering* = vertical organisation of services (each layer uses the one below). *Tiering* = physical distribution of layers over servers.	

(d) Design requirements for distributed architectures (➕ Coulouris)

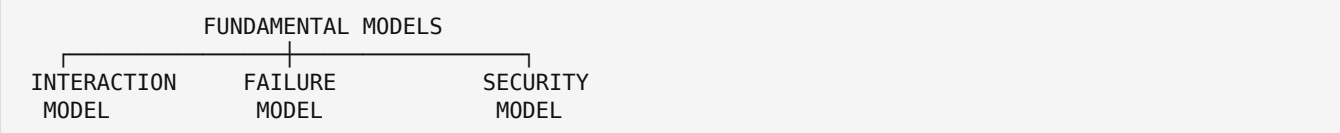
Performance (responsiveness, throughput, load balancing), **Quality of service** (reliability, security, adaptability), **Dependability** (correctness, fault-tolerance, security), caching & replication.

Exam-point table – architectures

Architecture	Key idea	Example
Client–server	Request/reply	Web
Multiple servers	Partition/replicate	DNS, Google
Proxy/cache	Nearby copies	Web proxy
Peer-to-peer	Equal roles	BitTorrent
Mobile code	Code moves to client	JavaScript
Mobile agent	Program moves around	Info-gathering agent
Thin client	Compute on server	Cloud desktop

5.3 FUNDAMENTAL MODELS

Definition: More **formal, abstract** descriptions of the **properties common to all architectures**. Three models:



(i) Interaction Model

Deals with **how processes interact (messages), communication performance, and time**.

- **Performance of communication channels:**
- **Latency** = delay between start of transmission at sender and start of receipt at receiver (network delay + OS delay + queueing).
- **Bandwidth** = total volume of data transmittable per unit time.
- **Jitter** = variation in time to deliver a series of messages (important for audio/video).
- Transmission time ≈ **latency + (message length / bandwidth)**.
- **Computer clocks & timing:** each process has own clock; **clock drift rate** = how fast clock differs from perfect reference; no global time.
- **Two variants of the interaction model:**

Synchronous DS	Asynchronous DS
Known bounds on: process execution time, message delivery time, clock drift rate	**No** bounds on any of these
Timeouts can detect failures reliably	Cannot know if process is slow or crashed
Easier to design algorithms	Realistic (Internet); harder
e.g., real-time embedded systems	e.g., the Internet

- **Event ordering:** Because there's no global clock, we order events by **causality** – logical clocks (Unit-II).

(ii) Failure Model

Defines **how failures can occur** so we can understand their effects.

Class	Sub-type	Description
Omission failures	Process omission (crash)	Process stops and never resumes (fail-stop if others can detect it)
	Channel omission	Message lost: send-omission, receive-omission, channel omission (buffer overflow, corruption)
Arbitrary (Byzantine) failures	–	Process/channel behaves arbitrarily: wrong values, sends contradicting messages, corrupts messages – worst kind
Timing failures	Clock, performance	Response takes longer than allowed (only in synchronous systems)

- **Masking failures:** hide failure or convert to a more acceptable kind (e.g., checksum turns corruption into omission; replication).
- **Reliable communication** = **validity** (every message sent is eventually delivered) + **integrity** (message received is identical to that sent and no duplicates).

(iii) Security Model

Ensures **secure processes and channels** and protects objects. - **Objects & principals:** resources have *access rights*; users/servers = *principals*. - **Threats:** to processes (spoofing/forging identity), to channels (eavesdropping, tampering, replay), **denial of service**, mobile code threats. - **Defences:** cryptography (encryption), **authentication**, **access control**, secure channels (SSL/TLS), digital signatures. - The "enemy" can send/copy/read/modify messages.

Comparison: Architectural vs Fundamental

Architectural	Fundamental
Structure/organisation of components	Abstract properties & assumptions
Concrete (client-server, P2P…)	Abstract (interaction, failure, security)
Answers "how is it built?"	Answers "how does it behave / what can go wrong?"

🧠 Remember This

> Architectural = **who is placed where** (CS, P2P, proxy, mobile code). Fundamental = **Interaction (sync/async, latency, drift) + Failure (omission, arbitrary, timing) + Security (threats, crypto)**.

---

6. NETWORKING AND INTERNETWORKING 📘📝 (A1-2b)

Definition

- **Network:** Set of interconnected devices that exchange data using communication protocols.
- **Internetwork (internet):** A **network of networks** – several different networks connected by **routers/gateways** so that they behave as a single logical network (e.g., the Internet).

Role in Distributed Systems

Distributed systems rely on networks to carry **messages** between components. Networks give **connectivity** (reach), while protocols give **rules** (format, order, error control) so heterogeneous machines can talk.

Types of networks (➕ by scale)

Type	Range	Example	Speed/Latency
PAN	few metres	Bluetooth	–
LAN	building/campus	Ethernet, Wi-Fi	High speed, low latency

Type	Range	Example	Speed/Latency
MAN	city	Cable TV / metro Ethernet	Medium
WAN	country/world	Internet backbone	Slower, higher latency
Wireless (WLAN/WMAN/WWAN)	varied	Wi-Fi, 4G/5G	Lossy, mobile

Key networking concepts (➕)

- **Packet switching:** data broken into packets, each routed independently (vs circuit switching).
- **Protocol:** agreed rules for communication. **Protocol suite/stack:** layered set of protocols.
- **Layering** – OSI 7 layers: Physical, Data link, Network, Transport, Session, Presentation, Application. **TCP/IP** (4 layers): Link, Internet (IP), Transport (TCP/UDP), Application (HTTP, FTP, SMTP, DNS).

```
Application layer   HTTP FTP SMTP DNS      ← messages
Transport layer     TCP / UDP              ← segments (end-to-end, ports)
Internet layer      IP (addressing/routing)← packets
Link/Physical       Ethernet, Wi-Fi        ← frames/bits
```

- **TCP vs UDP:** | TCP | UDP | |---|---| | Connection-oriented, reliable, ordered | Connectionless, unreliable, no ordering | | Flow & congestion control | None | | Slower, more overhead | Faster, low overhead | | Web, e-mail, file transfer | DNS, video/voice streaming, games |
- **IP addressing & routing:** each host has IP address; **routers** forward packets between networks; **gateways** connect networks using different protocols; **switches/bridges** connect within LAN.
- **Internetworking devices:** repeater (L1), hub, bridge/switch (L2), **router** (L3), gateway (all layers).
- **Performance parameters: latency, bandwidth, jitter**, reliability, scalability, security, mobility, **QoS**.

Internetworking diagram

```
LAN-1 (Ethernet)          LAN-2 (Wi-Fi)
[PC]─[PC]─┐                ┌─[Laptop]
          [Router]═WAN═[Router]
[Server]──┘                └─[Phone]
```

Exam points

- Write: network → internet → protocols → layering → TCP/UDP → link to DS (messages, latency, failures).
- Networks are **not reliable, not secure, have latency, bandwidth limits, are heterogeneous** – the **fallacies of distributed computing** ➕.

🧠 Remember This

> Internetwork = network of networks joined by routers. Protocol layers give uniform communication. TCP = reliable, UDP = fast.

---

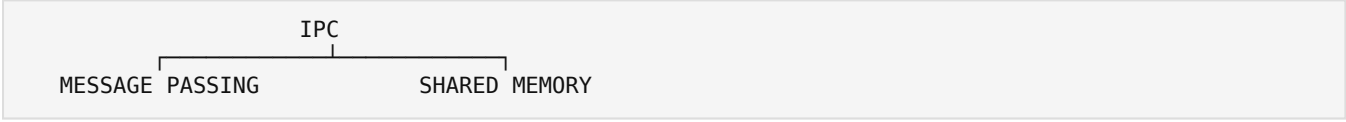
7. INTERPROCESS COMMUNICATION (IPC) 📘📝 (A1-3a)

Definition

IPC = mechanisms that allow **processes to exchange data and synchronise** their actions, whether on one computer or on different computers.

Simple (🗣️)

Do processes ko baat karni hai – ya to **chitthi (message)** bhejo, ya **ek common notice-board (shared memory)** use karo.



7.1 Message Passing

Definition: Processes communicate by **explicitly sending and receiving messages** via a communication channel (no shared variables). Two primitives: `send(destination, message)` and `receive(source, message)`.

Concepts ➕ (Coulouris ch. 4)

Concept	Options
Synchronous (blocking)	Sender blocks until receiver has received; receiver blocks until message arrives
Asynchronous (non-blocking)	Sender continues after copying message to buffer; receive may be blocking or non-blocking (polling/interrupt)
Reliability	Reliable channel (TCP) vs unreliable (UDP)
Ordering	FIFO ordering
Addressing	Sockets = (IP address, port); destination via port
Direct/Indirect	Direct: to process; Indirect: via **mailbox/message queue**
Buffering	Zero, bounded or unbounded buffer

Communication styles/mechanisms: - **Sockets:** endpoint for communication – **UDP datagram sockets** (`DatagramSocket`) and **TCP stream sockets** (`Socket`, `ServerSocket`). -

Message queues, MPI ➕ (parallel), **request–reply protocol**, RPC/RMI (higher-level). - **Marshalling/unmarshalling** (➕): converting data structures to a flat sequence of bytes (external data representation: CORBA CDR, Java serialization, XML, JSON, Protocol Buffers) so heterogeneous machines can understand.

Diagram

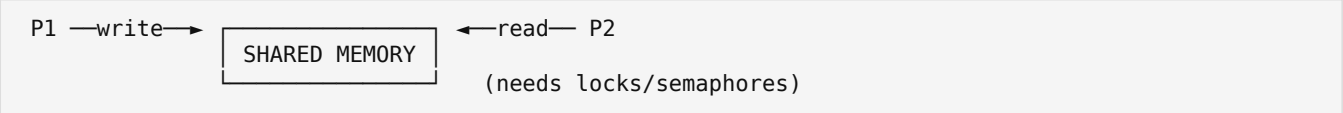


Advantages: works between different computers, no shared memory needed, clear synchronisation, portable, secure/isolated, suitable for distributed systems. **Disadvantages:** slower (network/copy/marshalling), programmer handles message format & failures, overhead of system calls. **Examples:** sockets, e-mail, chat, MPI, RabbitMQ/Kafka.

7.2 Shared Memory

Definition: Processes communicate by **reading and writing a common memory region** that both can access. Needs **synchronisation** (locks/semaphores) to avoid race conditions.

- On a **single machine:** OS provides shared memory segments (`shmget`, `mmap`), threads share address space.
- In a **distributed system:** no physical shared memory → emulate it via **Distributed Shared Memory (DSM)** ➕ – a software layer giving an illusion of one shared address space using page faults & message passing underneath.



Advantages: very **fast** (no copy after set-up), easy for large data, programming simple (like normal variables). **Disadvantages:** only on same machine (or emulated with DSM), **race conditions / need synchronisation**, poor security/isolation, DSM has coherence issues & false sharing, less scalable. **Examples:** threads in one process, OS shared memory (POSIX), DSM systems (Ivy, TreadMarks).

7.3 COMPARISON: Message Passing vs Shared Memory 📝

Point	Message Passing	Shared Memory
Communication	Explicit `send`/`receive`	Read/write to common variables
Location	Same or **different computers**	Same computer (or DSM emulation)
Speed	Slower (copying, network, system calls)	Faster (direct memory access)
Synchronisation	Implicit in blocking send/receive	Explicit (locks, semaphores, monitors)
Data transfer	Copied between address spaces	No copy, data shared in place
Complexity	Programmer handles message format/failures	Programmer handles race conditions

Point	Message Passing	Shared Memory
Scalability	**High** (suitable for distributed)	Low (limited to few processors)
Fault isolation/Security	Better isolation	Poor: one bug corrupts shared data
Suitable for	Distributed systems, loosely coupled	Multiprocessors, tightly coupled
Implementation	Sockets, MPI, queues, RPC/RMI	shm, mmap, threads, DSM
Examples	Chat, client-server	Multi-threaded app, producer–consumer buffer

Exam points

- Concluding line: "Distributed systems use **message passing** because there is no physical shared memory among nodes."
- Also mention: **request–reply protocol** is built on message passing and underlies RPC/RMI.

🧠 Remember This

> Message passing = explicit send/receive, works over network, slower, scalable. Shared memory = common memory, fast, same machine, needs locks.

---

8. DISTRIBUTED OBJECTS 📘📝 (A1-3b)

Definition

A **distributed object** is an object (data + methods) whose **methods can be invoked by processes located on other computers**, as if it were local. The **object model** is extended to work across the network.

Simple (🗣️)

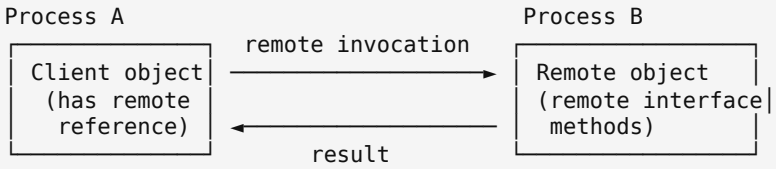
Object ek computer pe hai, uske method ko dusre computer se call karna – bilkul local method jaisa.

Key terms

Term	Meaning
Remote object	Object whose methods can be invoked from another process
Remote object reference	Identifier (ID) used to refer to a remote object: internet address + port + time + object number + interface; passed as argument/result
Remote interface	Specifies which methods of a remote object can be invoked remotely (like a contract); defined in IDL (CORBA) or Java interface extending `Remote`

Term	Meaning
Local vs remote invocation	Remote: arguments/results **marshalled**; failures possible; different semantics (pass by value for objects, by reference for remote objects)
Actions	Invocation may cause state change or invocation of further objects (chain of invocations)
Exceptions	Extra `RemoteException` for communication failures
Garbage collection	Automatic deletion of objects no longer referenced remotely (reference counting / leases)

Distributed Object Model – diagram



Advantages

Object-oriented benefits (encapsulation, reuse), location transparency, easier design of complex distributed apps, interoperability (CORBA).

Disadvantages

Performance overhead, complexity of failures, garbage collection, tight coupling to language (Java RMI) or heavy middleware (CORBA).

Systems ➕: Java RMI, CORBA, DCOM, .NET Remoting.

🧠 Remember This

> Distributed object = object usable from other computers through **remote reference + remote interface**.

---

9. REMOTE METHOD INVOCATION (RMI) 📘📝 (A1-3b, A1-1c)

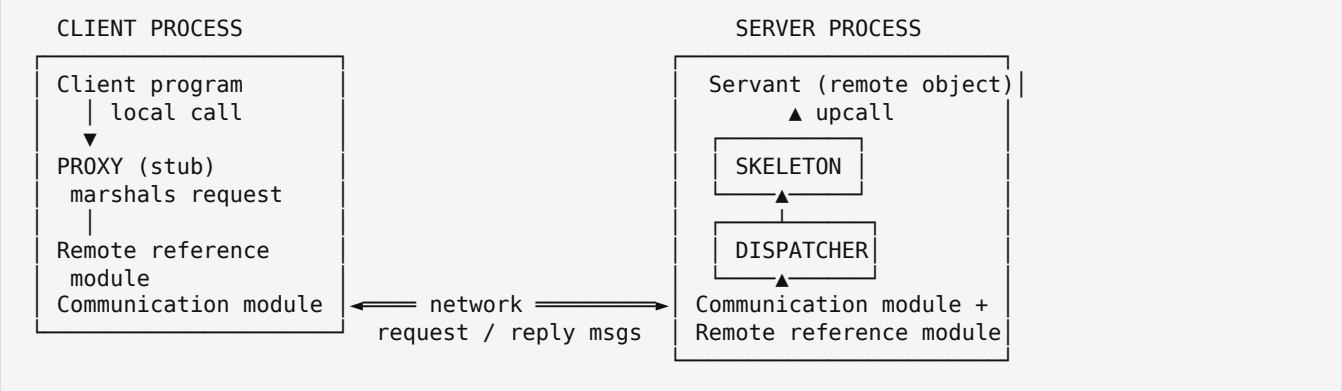
Definition

RMI allows an object running in one JVM/process to **invoke methods on an object in another process** (usually on another machine) using the same syntax as a local call – it is the **object-oriented version of RPC**.

Simple (🗣️)

`result = remoteObj.add(5,3);` – dikhta local call hai, par execute dusre computer pe hota hai. Beech ka kaam **stub/proxy** karta hai.

RMI software architecture (Coulouris 4th ed.)

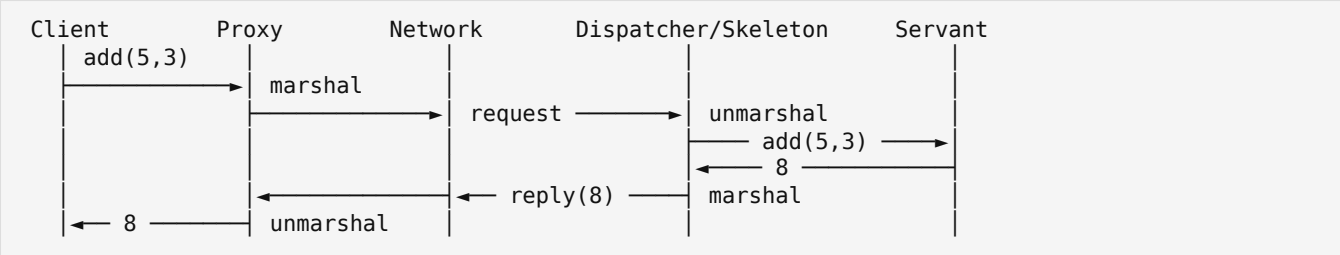


Component	Role
Proxy (Stub)	Local representative of remote object at client; **marshals** method id + arguments into request, sends it, waits, **unmarshals** result; hides remote nature (transparency)
Communication module	Implements request–reply protocol between client & server; uses invocation semantics
Remote reference module	Translates between local and remote object references; maintains **remote object table**
Dispatcher	Server side; receives request, selects the method (using method id) and passes to skeleton; one per class of remote object
Skeleton	Unmarshals arguments, calls the method on the servant, **marshals** result/exception and returns
Servant	The actual implementation object that does the work
Binder / Registry	Naming service mapping textual names to remote references (RMI registry)

Working – steps of a remote method call

1. Client calls method on **proxy** (looks local).
2. Proxy **marshals** (method id + parameters) into a message.
3. **Communication module** sends the request message to the server (using TCP/IP).
4. Server's communication module receives it; **remote reference module** finds the target object.
5. **Dispatcher** selects the correct skeleton/method.
6. **Skeleton unmarshals** arguments and **invokes** the servant method.
7. Servant returns result → skeleton **marshals** the result.
8. Reply sent to client; proxy **unmarshals** and returns result to the caller.

Sequence diagram



Invocation semantics ➕ (fault tolerance measures)

Semantics	Retransmit request?	Filter duplicates?	Re-execute/ retransmit reply	Result
Maybe	No	–	–	Method executed once or not at all (no guarantee)
At-least-once	Yes	No	Re-execute procedure	Executed ≥ 1 times; OK for **idempotent** ops
At-most-once	Yes	Yes	Retransmit old reply	Executed exactly once or not at all; **Java RMI** provides this

Advantages

Object-oriented, transparency, easy programming, type checking, passes objects (serialisation), garbage collection, secure (Java security manager).

Disadvantages

Slower than local call, **Java-only** (Java RMI), firewall issues, failures more complex, parameter passing overhead.

Exam points

- Draw the proxy–skeleton diagram; list 8 steps.
- Mention remote reference passing and parameter passing: **objects by value (serialised), remote objects by reference**.

🧠 Remember This

> RMI = call object's method remotely. Client → **stub/proxy** → network → **skeleton/dispatcher** → **servant** → result back.

---

10. RPC — REMOTE PROCEDURE CALL 📘

Definition

RPC lets a program **call a procedure that runs in another address space (another computer)** as though it were a local procedure call; the network details are hidden (Birrell & Nelson, 1984).

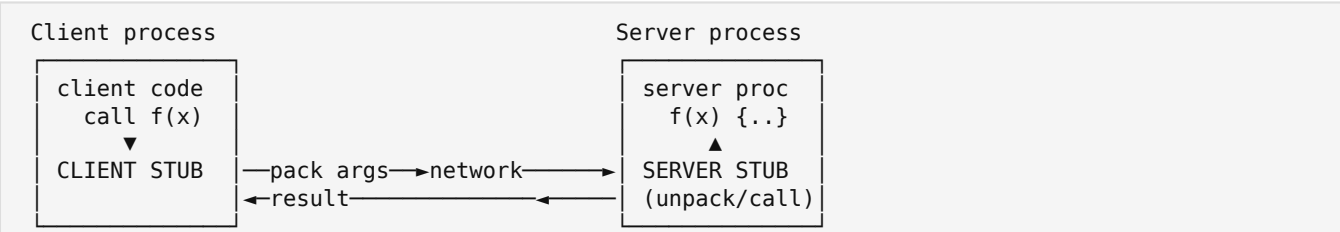
Simple (🗣️)

Function call jaisa likho, execution dusre computer pe. Beech me **client stub** aur **server stub** sab sambhal lete hain.

Components

Component	Role
Client	Calls the procedure
Client stub	Marshals (packs) parameters into message; unpacks result
RPC runtime / communication	Sends/receives messages, retransmission, timeouts
Server stub (skeleton)	Unpacks parameters; calls actual procedure; packs result
Server procedure	The real function
IDL	Interface Definition Language – describes procedure signatures; **IDL compiler generates stubs**
Binder/Portmapper	Locates server (name service)
XDR / data representation	Common format for heterogeneous machines

Diagram



Working – steps

1. Client calls local **client stub** (normal call).
2. Stub **marshals parameters** into message.
3. Message sent to server (via RPC runtime, OS, network).
4. **Server stub unmarshals** the parameters.
5. Server stub calls the **actual procedure**.
6. Procedure executes and returns result to server stub.
7. Server stub **marshals** result and sends back.
8. Client stub **unmarshals** the result and returns to the client program.

Issues in RPC ➕

Parameter passing (**no pointers/references**: pass by value/copy-restore), data representation (endianness, XDR), binding, failures (lost request/reply, server/client crash, orphan), **semantics** (maybe, at-least-once, at-most-once), performance, security, no shared global variables.

Advantages

Simple & familiar programming model, hides network, language-independent via IDL, modular/reusable, widely used (Sun RPC/ONC RPC, gRPC).

Disadvantages

Slower than local calls, failure handling is complex (partial failure), no pointers, tight coupling, less flexible than messaging, security concerns.

Examples ➕: Sun/ONC RPC (used by NFS), DCE RPC, gRPC, XML-RPC, Apache Thrift.

10.5 COMPARISON: RMI vs RPC 📝

Feature	RPC	RMI
Paradigm	**Procedural** (functions)	**Object-oriented** (methods of objects)
What is invoked	Remote procedure/function	Method of a remote object
Parameters	Ordinary data structures (by value); **no object passing**, no pointers	Can pass **objects** (serialised, by value) and **remote references**
Language	Mostly language-independent (C, via IDL/XDR)	Java RMI is **Java-specific** (CORBA IIOP is multi-language)
Stubs	Client stub / server stub	Stub (proxy) / skeleton
Interface definition	IDL (e.g., rpcgen)	Java interface extending `Remote`
Object identity	Not applicable	Remote object references, **garbage collection**
Polymorphism/inheritance	Not supported	Supported (dynamic loading of classes)
Complexity/overhead	Simpler, lighter	Heavier (objects, serialisation, registry)
Naming	Portmapper / binder	RMI Registry / naming service
Typical use	System services (NFS)	Enterprise Java apps

Exam point: "RMI is the OO counterpart of RPC: both hide remote communication behind a local call using stubs and marshalling; RMI adds object references, object parameters and garbage collection."

🧠 Remember This

> RPC = call **function** remotely (procedural). RMI = call **method of object** remotely (OO). Both use stub + marshalling.

---

11. EVENTS AND NOTIFICATIONS 📘

Definition

- **Event:** an occurrence of interest at an object (e.g., button click, file modified, price change, sensor reading).
- **Notification:** a message/object containing information about an event, sent to interested parties.
- **Publish–Subscribe (distributed event-based system):** *publishers* generate events; *subscribers* register interest; an **event service** delivers notifications **asynchronously** to all interested subscribers.

Simple (🗣️)

YouTube channel: creator video upload karta hai (event), jinhone **subscribe** kiya hai unko **notification** milti hai. Subscriber ko baar-baar poochna nahi padta.

Roles in the model (Coulouris)

Role	Function
Event generator / Publisher	Object of interest that changes state and generates events
Event type	Kind of event, with attributes
Subscriber / Observer / Listener	Registers interest; receives notification
Event service (broker)	Middleman that stores subscriptions, filters, and delivers notifications
Observer (in the Coulouris sense)	Object that processes notifications: forwards, filters, patterns, logs (decouples subscriber from publisher)

Diagram



Characteristics

- **Heterogeneous** (different systems interoperate), **asynchronous**, **decoupled** in space, time (and sync) – publisher doesn't know subscribers.
- Types of subscription: **channel/topic-based, content-based, type-based**.

Working steps

1. Subscribers register (subscribe) for event types with the event service.
2. Publisher publishes an event/notification.
3. Event service matches it with subscriptions.
4. Notification delivered to all matching subscribers.
5. Subscribers act; can unsubscribe.

Architecture/Delivery issues ➕: reliability (guaranteed delivery), ordering, scalability, filtering, security, event persistence (e.g., Kafka, MQTT).

Advantages: loose coupling, scalable, asynchronous, easy to add subscribers, real-time updates.

Disadvantages: delivery/ordering hard to guarantee, hard to debug, broker may be bottleneck/SPOF, security/privacy.

Examples: stock ticker, push notifications, Java `EventListener`, RabbitMQ/Kafka/MQTT, Jini distributed events, GUI events, Facebook notifications, IoT sensors.

🧠 Remember This

> Publisher → **Event Service** → Subscribers. Asynchronous, loosely-coupled notifications about events.

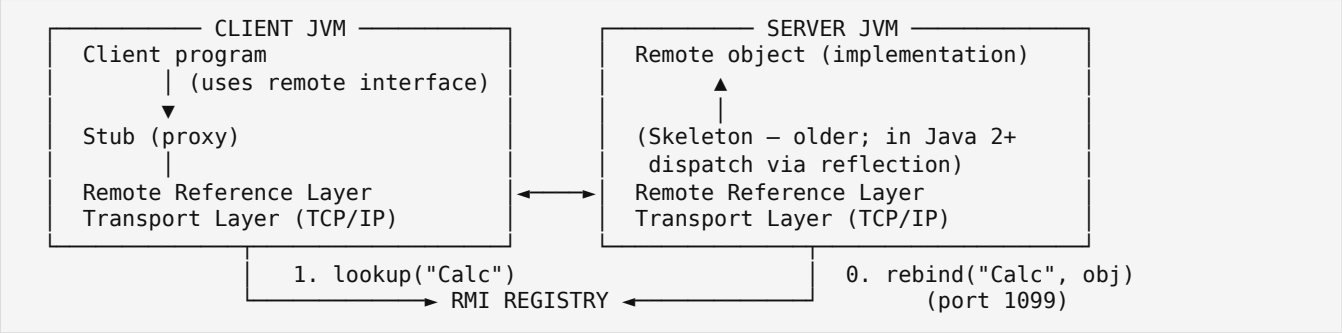
---

12. CASE STUDY: JAVA RMI 📘📝 (A1-1c – 3 marks, the biggest Unit-I question)

12.1 What is Java RMI?

Java RMI is Java's mechanism (package `java.rmi`) for **invoking methods of an object residing in another JVM** (same or different machine). It follows the distributed-object model, is **Java-to-Java**, supports **at-most-once** semantics, serialisation for object passing, and distributed garbage collection.

12.2 Architecture



Three layers: (1) **Stub/Skeleton layer** – intercepts calls, marshals; (2) **Remote Reference Layer** – manages references & invocation semantics; (3) **Transport layer** – TCP connections between JVMs.

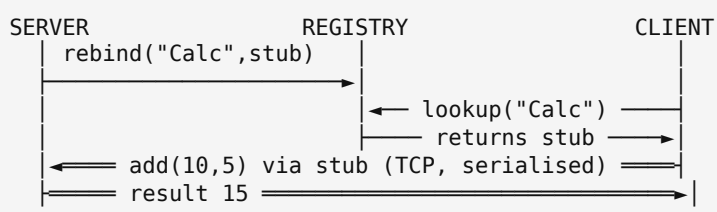
12.3 Components

Component	Explanation
Remote interface	A Java interface that **extends** `java.rmi.Remote`; each method **declares** `throws RemoteException`. Defines what client can call.
Remote object (implementation)	Class implementing the remote interface; usually **extends** `UnicastRemoteObject` (exports the object so it can receive calls). Contains actual logic.
Stub (proxy)	Client-side representative; implements same remote interface; forwards call to server. Since Java 5 generated **dynamically** (no `rmic` needed); earlier created with `rmic`.
Skeleton	Server-side entity that receives call and invokes method; **obsolete since Java 1.2** (handled by RMI runtime). Mention for exam completeness.
RMI Registry	Simple name service (`rmiregistry`, default **port 1099**) mapping name → stub. Server **binds/rebinds**; client **looks up**.
Server	Creates remote object, exports it, registers in registry.
Client	Looks up name in registry, gets stub, calls methods.
Serialization	Objects passed as parameters must implement `java.io.Serializable`.

12.4 Complete execution flow (step-by-step)

1. **Define** remote interface (`Calculator extends Remote`).
2. **Implement** it (`CalculatorImpl extends UnicastRemoteObject`).
3. **Server** starts: creates object → **exports** (stub generated) → starts/gets registry → **`rebind("CalculatorService", obj)`**.
4. **Client** starts: contacts registry host:1099 → **`lookup("CalculatorService")`** → receives **stub**.
5. Client **calls** `stub.add(10,5)`.
6. Stub **marshals (serialises)** method + arguments → sends over TCP.
7. Server-side RMI runtime **unmarshals**, invokes the method on the **real object**.
8. Result **marshalled** and returned; stub **unmarshals** and gives value to client.

9. Client prints result. Unused objects are **garbage-collected** through leases/reference counting.



12.5 Simple Calculator example

Step 1 – Remote interface `Calculator.java`

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}
```

Step 2 – Implementation (remote object) `CalculatorImpl.java`

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }

    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }
    public double divide(int a, int b) { return (double) a / b; }
}
```

Step 3 – Server `CalculatorServer.java`

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            Calculator calc = new CalculatorImpl();          // create remote object
            Registry registry = LocateRegistry.createRegistry(1099); // start registry
            registry.rebind("CalculatorService", calc);       // register name -> stub
            System.out.println("Calculator Server is ready...");
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Step 4 – Client `CalculatorClient.java`

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calc = (Calculator) registry.lookup("CalculatorService");
            System.out.println("10 + 5 = " + calc.add(10, 5));
            System.out.println("10 - 5 = " + calc.subtract(10, 5));
            System.out.println("10 * 5 = " + calc.multiply(10, 5));
            System.out.println("10 / 5 = " + calc.divide(10, 5));
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Step 5 – Compile & run

```
javac *.java
java CalculatorServer        (terminal 1)
java CalculatorClient        (terminal 2)
```

Sample output

```
Server terminal :  Calculator Server is ready...
Client terminal :  10 + 5 = 15
                   10 - 5 = 5
                   10 * 5 = 50
                   10 / 5 = 2.0
```

> Notes: (i) `Naming.rebind("rmi://localhost/CalculatorService", calc)` is an alternative. (ii) With old Java (< 5) run `rmic CalculatorImpl` to generate stub. (iii) The server JVM stays alive because the exported object keeps it running.

12.6 Advantages / limitations of Java RMI

Advantages	Limitations
Simple, pure Java, object-oriented	Java-only (no cross-language)
Automatic stub, serialisation, distributed GC	Slower than local call / heavier than sockets
Dynamic class loading, security manager	Firewall issues (dynamic ports)
Type-safe, passes objects	Registry is simple (no advanced naming)
At-most-once semantics	Objects must be serializable

12.7 Viva questions – Java RMI

1. **What is Java RMI?** Java API to call methods of objects in another JVM as if local.
2. **Which interface must a remote interface extend?** `java.rmi.Remote`.
3. **Why does every remote method throw `RemoteException`?** Network can fail; caller must handle communication errors.
4. **Role of the stub?** Client-side proxy; marshals calls and sends them to server.
5. **What is the RMI registry and its default port?** Naming service, port 1099.

6. **Difference between `bind` and `rebind`?** `bind` throws `AlreadyBoundException` if name exists; `rebind` overwrites.
7. **What does `UnicastRemoteObject` do?** Exports the object (creates stub, listens on port) for remote calls.
8. **How are parameters passed?** Primitives/objects by value (serialised); remote objects by reference (stub).
9. **What is a skeleton?** Server-side dispatcher of older RMI; not needed since Java 1.2.
10. **Which invocation semantics does Java RMI give?** At-most-once.
11. **What is marshalling?** Converting arguments to a byte stream for transmission.
12. **How is a remote object garbage collected?** Distributed GC using reference counting & leases.

🧠 Remember This

> Java RMI = Remote interface (extends Remote) + Impl (extends UnicastRemoteObject) + Server (rebind in registry) + Client (lookup → stub → call).

(End of Unit-I. Next: `02_Unit2_Notes.md`)